G16 Project 3 Documentation

1. Project Summary

G16 Project 3 is a multithreaded file management server written in C. It exposes a RESP-compatible TCP protocol, so `redis-cli` can be used directly as a client.
The server supports core file operations, metadata queries, audit log retrieval, and simple RLE-based compression/decompression, while enforcing path safety and per-file synchronization.

2. Architecture

Core modules:

- `src/server.c` : socket setup, client session handling, command dispatch.
- `src/resp.c` , `src/resp.h` : RESP request parsing and response serialization.
- `src/file_ops.c` , `src/file_ops.h` : command handlers for file operations.
- `src/locks.c` , `src/locks.h` : lock registry for per-path read/write synchronization.
- `src/path_utils.c` , `src/path_utils.h` : canonicalization and safe path checks.
- `src/logger.c` , `src/logger.h` : thread-safe operation logging.

3. Supported Commands

- `PING [msg]`
- `GETFILE <path>`
- `SETFILE <path> <content>`
- `DELFILE <path>`
- `RENAMEFILE <old_path> <new_path>`
- `COPYFILE <src_path> <dst_path>`
- `STATFILE <path>`
- `LOGS [line_count]`
- `COMPRESS <src_path> [dst_path]`
- `DECOMPRESS <src_path> [dst_path]`

4. Build and Run

Build:

```
make
```

Run:

```
./filesrv -p 6380 -d ./data
```

Sample client usage:

```
redis-cli -p 6380 PING
redis-cli -p 6380 SETFILE notes/hello.txt "hello"
redis-cli -p 6380 GETFILE notes/hello.txt
redis-cli -p 6380 LOGS 20
```

5. Concurrency and Safety Notes

- Locking is done per canonical file path.
- Multi-path commands lock both paths in sorted order to avoid deadlocks.
- Unsafe paths are rejected (`..` and absolute paths are not allowed).
- Audit events are written to `<data_dir>/operations.log` .

6. Compression Format

- Uses run-length encoding with `RLE1` header.
- Default outputs:
- `COMPRESS file.txt` -> `file.txt.rle`
- `DECOMPRESS file.txt.rle` -> `file.txt`
- Non- `.rle` decompression source defaults to `<source>.out`

7. Contribution Breakdown (Who Contributed What)

Attribution below is based on git history for files inside this project.

Rachit Kumar Pandey

- `src/server.c`
- `src/file_ops.h`
- `Makefile`
- Core server bootstrap/integration and build wiring.

Rahul Joshi

- `src/file_ops.c`
- `README.md`
- File-operation implementation and project-level usage documentation.

Ramala Karthik

- `src/resp.c`
- `src/logger.c`
- RESP processing and logging implementation.

Ranish Garg

- `src/resp.h`
- `src/logger.h`
- RESP and logger interface design (header-level contracts).

Raj Priyadarshi

- `src/path_utils.c`
- `src/locks.c`
- Path validation/canonicalization logic and lock-system implementation.

`Rajarshi Chakraborty`

- `src/path_utils.h`
- `src/locks.h`
- Header/interface definitions for path and lock modules.

Implemented commands:

- `PING [msg]`
- `GETFILE <path>`
- `SETFILE <path> <content>`
- `DELFILE <path>`
- `RENAMEFILE <old_path> <new_path>`
- `COPYFILE <src_path> <dst_path>`
- `STATFILE <path>`
- `LOGS [line_count]`
- `COMPRESS <src_path> [dst_path]`
- `DECOMPRESS <src_path> [dst_path]`

Compression format:

- RLE with `RLE1` header.
- If destination is omitted:

- `COMPRESS file.txt` writes to `file.txt.rle`.
- `DECOMPRESS file.txt.rle` writes to `file.txt`.
- If source does not end with `.rle`, decompression default output is `<source>.out`.

8. Screenshot Placeholders

- Build output screenshot:

```
→  G16_Project_3 git:(main) ✗ make
gcc -std=c11 -D_XOPEN_SOURCE=700 -D_DEFAULT_SOURCE -Wall -Wextra -Wpedan
tic -O2 -pthread -o filesrv src/server.c src/resp.c src/locks.c src/path
_utils.c src/logger.c src/file_ops.c
→  G16_Project_3 git:(main) ✗ ./filesrv
filesrv listening on 0.0.0.0:6379, data dir: .
```

```
→  G16_Project_3 git:(main) ✗ redis-cli PING
PONG
→  G16_Project_3 git:(main) ✗ redis-cli
127.0.0.1:6379>
```

- RESP command demo screenshot:

```
➜  G16_Project_3 git:(main) ✗ redis-cli
127.0.0.1:6379> SETFILE test_file "Hello world"
OK
127.0.0.1:6379> GETFILE test_file
"Hello world"
127.0.0.1:6379> DELFILE test_file
(integer) 1
127.0.0.1:6379> GETFILE test_file
(nil)
127.0.0.1:6379>
```

- Logs/operations screenshot:

```
127.0.0.1:6379> SETFILE test_file "Hello world"
OK
127.0.0.1:6379> GETFILE test_file
"Hello world"
127.0.0.1:6379> DELFILE test_file
(integer) 1
127.0.0.1:6379> GETFILE test_file
(nil)
127.0.0.1:6379> LOGS 5
"2026-04-14 22:18:36 client=127.0.0.1 cmd=SETFILE target=test_file statu
s=OK detail=write success\n2026-04-14 22:18:41 client=127.0.0.1 cmd=GETF
ILE target=test_file status=OK detail=read success\n2026-04-14 22:18:46
client=127.0.0.1 cmd=DELFILE target=test_file status=OK detail=deleted\n
2026-04-14 22:18:50 client=127.0.0.1 cmd=GETFILE target=test_file status
=MISS detail=not found\n"
127.0.0.1:6379>
```

  - Copy file and file stat:

```
127.0.0.1:6379> SETFILE test_file "The quick brown fox jumps over the la
zy dog"
OK
127.0.0.1:6379> GETFILE test_file
"The quick brown fox jumps over the lazy dog"
127.0.0.1:6379> COPYFILE test_file test_copy
OK
127.0.0.1:6379> STATFILE test_copy
"size=43 mode=644 uid=1000 gid=984 atime=1776185398 mtime=1776185398 cti
me=1776185398"
127.0.0.1:6379>
```

- File compression/decompression:

```
→  G16_Project_3 git:(main) × cat test_compress.rle
RLE1ABAC%
→  G16_Project_3 git:(main) × cat test_compress
AAAAABBAAAACCCCCC%
→  G16_Project_3 git:(main) ×
```

```
→  G16_Project_3 git:(main) × make
gcc -std=c11 -D_XOPEN_SOURCE=700 -D_DEFAULT_SOURCE -Wall -Wextra -Wpedantic -O2 -pthread -o filesrv src/server.c src/resp.c src/locks.c src/path_utils.c src/logger.c src/file_ops.c
→  G16_Project_3 git:(main) × ./filesrv
filesrv listening on 0.0.0.0:6379, data dir: .
```

```
127.0.0.1:6379> SETFILE test_compress "AAAAABBAAAACCCCCC"
OK
127.0.0.1:6379> COMPRESS test_compress
OK
127.0.0.1:6379> GETFILE test_compress
"AAAAABBAAAACCCCCC"
```

9. Known Limitations

- Compression is simple RLE and may not reduce size for high-entropy data.
- Functionality is scoped to a configured data directory (by design).
- Access control/authentication is not part of the current protocol scope.